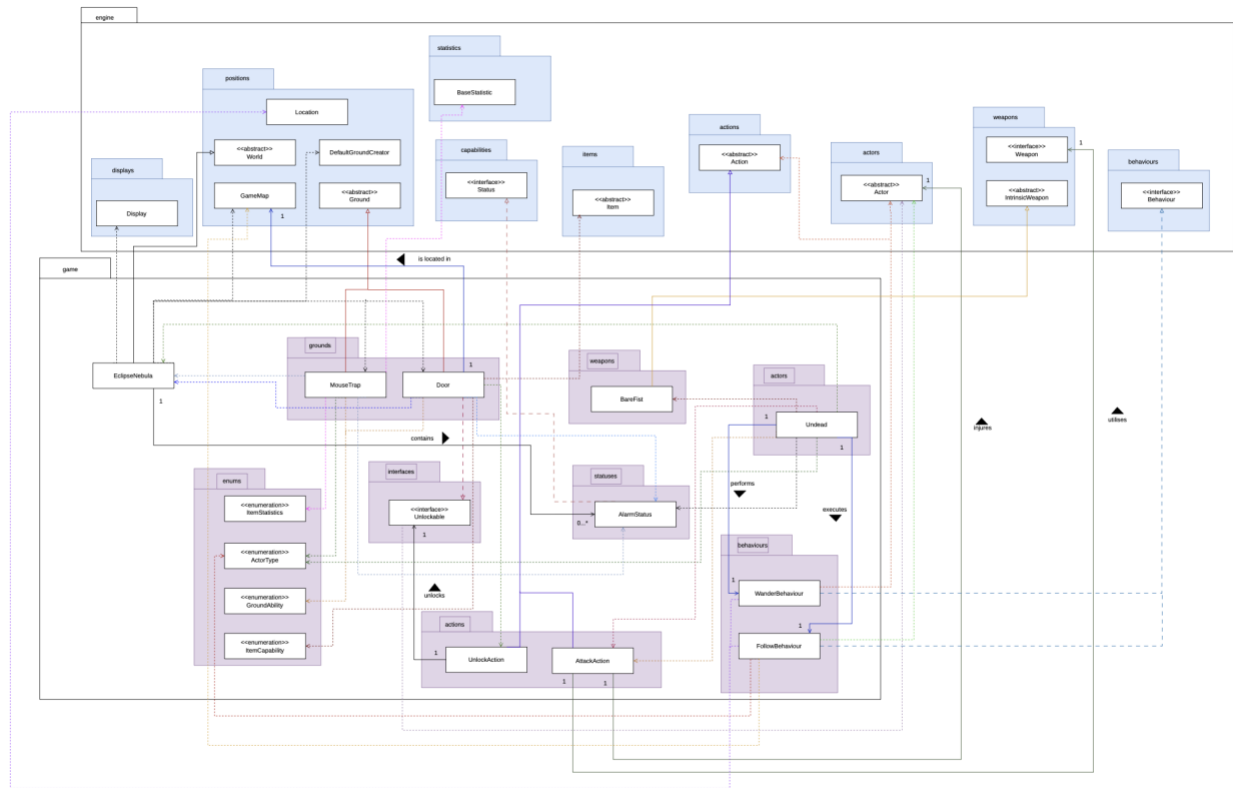






REQ 3 UML Class Diagram:





Design Rationale

REQ 1

A. Implementation of Consumable System

Design 1: Create a Consumable interface which can be implemented by items that are consumed by the actor.

Pros	Cons
Easier to define own consumption behaviour for each item and supports additional consumable types without modifying existing classes.	Similar attributes can't be grouped together and would need to have their own set of attributes initialised in the respective concrete item classes.
Enforce contract for consumable items to have the same set of methods, but able to have different effects for different items, since interfaces do not define the method.	More repeated code, methods enforced that have the same logic would still need to be implemented in all concrete classes that use the interface.

Design 2: Create a Consumable abstract class which extends Item from the engine and create different items that are consumable by extending this class.

Pros	Cons
Shared attributes and methods can be defined in the abstract class and items that are consumable just need to inherit these attributes and methods, which results in less repetition of code.	Each concrete class can only extend a single abstract class, which limits the development of the individual consumable item classes.
Changes to a common method that is needed by all consumable items can be modified in the single abstract class instead of changing individually in each concrete class.	

B. Implementation of Action logic for Items and Grounds

Design 1: Create a ConsumeAction and UnlockAction abstract class to handle interactions with objects that implement Consumable and Unlockable interfaces respectively.

Pros	Cons
Can be used for any object, regardless of whether it is an Item or Ground thus making it reusable.	Results in the creation of more classes.
The action logic is separated from the logic of each individual object, making the code cleaner.	Can be redundant for simpler implementations.

Design 2: Each class handles its own action related logic.

Pros	Cons
Simpler design with fewer classes required.	Each class will be performing/handling too many functionalities.
There won't be a need to handle additional Action objects.	Results in repetition of code logic.

C. Implementation of item weights

Design 1: Create a WeightedItem abstract class that extends the Item class and validates weight.

Pros	Cons
Reduces code repetition.	May be unnecessary if only a few items require weight handling.
Keeps weight handling consistent across all items.	Adds an extra layer of abstraction.
Ensures no negative weights are given to items.	

Design 2: Define weights in each individual item classes.

Pros	Cons
Simple and straightforward implementation logic.	Same logic is repeated across multiple classes.
There won't be any extra abstraction.	More prone to mistakes as more items are added.

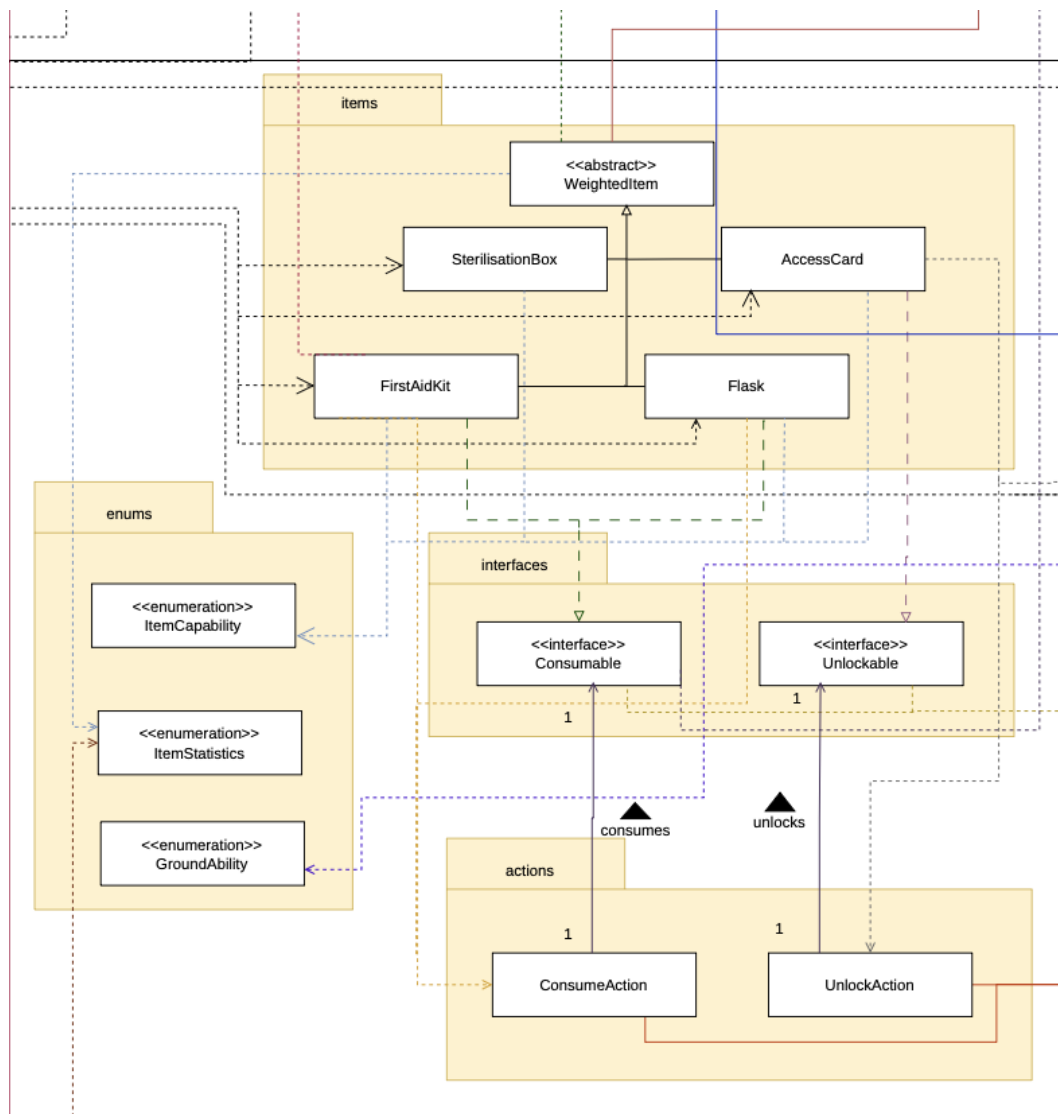
D. Determining item behaviour

Design 1: Create an ItemCapability enum which gives the ability that each item has, as a way of confirming the item's behaviour.

Pros	Cons
Easy to extend and maintain by adding new abilities for items.	Requires items to be correctly assigned their abilities.
Avoids checking specific classes directly.	Conditional checks are needed to verify the abilities.
The same logic can be reused across different item types.	

Design 2: Use instanceof to check the behaviour of each item.

Pros	Cons
Simple and straightforward to implement.	Leads to repeated type-checking and downcasting.
There would be no need to modify item structure or add additional abilities.	Tightly couples logic to specific item classes.



The diagram above shows the final design of requirement 1, which utilizes each Design 1 of parts A, B, C and design of part D stated above to implement the consumable system,

ConsumeAction and UnlockAction, creating a WeightedItem abstract class that extends Item and using an ItemCapability enum to check for the specific item.

Consumable and Unlockable interfaces are introduced as abstraction for items or grounds that can be consumed or unlocked respectively. Using these different interfaces for different traits ensure items or grounds can implement different behaviours (Interface Segregation Principle). Rather than placing the consumption and unlocking logic in each item class, the logic is delegated to a single ConsumeAction and UnlockAction classes respectively. This ensures each class is only responsible for a single aspect such as Flask only defining the Flask object and not having code related to the consuming action (Single Responsibility Principle). Since ConsumeAction and UnlockAction depend on the Consumable and Unlockable interfaces rather than concrete classes like Flask, the Dependency Inversion Principle is satisfied. Additionally, all items implementing Consumable or Unlockable can be substituted in ConsumeAction or UnlockAction without altering expected behaviour (Liskov Substitution Principle).

By implementing WeightedItem, all the item classes have a single unified source to help define the weight and characteristics of the item, where WeightedItem extends the Item class. This reduces duplication of weight initialization and validation logic across item subclasses, adhering to the DRY principle. This design remains simple and straightforward since the concrete item classes only need to call the parent class in the constructor to add their weights, which adheres to the KISS principle. Carrying this out eliminated the need for dependencies from each item class to ItemStatistics, wherein there is just a direct dependency from WeightedItem to ItemStatistics.

Lastly, by introducing ItemCapability enum rather than using instanceof, this design avoids tight coupling on specific concrete classes. This results in high cohesion as each class is focused on its own responsibilities (eg. ItemCapability purely defines capabilities, items purely define their own behaviour and actions purely handle execution logic).

The final design of Requirement 1 integrates interface-based behaviour action delegation a shared WeightedItem abstraction, and capability-based checks using ItemCapability. This combination ensures that item interactions are modular, reusable, and decoupled from concrete implementations. Overall, the design promotes flexibility and consistency by separating item behaviour, action execution, and attribute management into focused components.

REQ 2

- A. Changes to consumables to ensure safe consumption of those items

Design 1: Create a utility class ItemUtils which has a “hasSterilisationBox” method to check if the particular worker’s inventory contains a sterilization box.

Pros	Cons
Ensures consumables that don’t have poisoning effect don’t call on the method unless needed (eg. FirstAidKit).	Introduces dependency on external utility class.
Ensures consumable classes are only focused on their own behaviours and can be reused across any future classes that require sterilization check.	Additional class need for a very small feature.

Design 2: Create the method for sterilization check in the Consumable interface itself.

Pros	Cons
Easier to access for any consumable related implementation wherein only 1 class is called to, instead of needing additional helper classes.	Consumables that don’t require checking for sterilization box will be forced to use that method.
Direct access instead of require imports of utilities.	Can lead to redundant or unused method implementations.

B. Implementation of status effect systems

Design 1: Create a single DamageOverTimeStatus that handles all damage-over-time effects (e.g. poison, burn) using configurable values such as damage amount and duration.

Pros	Cons
Eliminates the need for different for multiple separate status classes.	Harder to extend unique behaviours and distinguish different effect types at a glance.
Prevents code duplication as all classes that have any damage involved can just call this class by creating a new instance of it, with the respective values.	Less expressive for different effect types.
Easier to apply consistent rules for timed damage effects.	

Design 2: Create dedicated status classes (eg. PoisonStatus, BurningStatus).

Pros	Cons
Allows each class to have unique and clearly defined behaviours.	Logic is repeated across multiple classes, since our system doesn't have any specific effect and only focuses on the damage and number of turns.
Easy extendable in the future if very specific effect implementations are required.	Hard to maintain when many objects with effects exist as many different classes will be introduced.

C. Implementation of the environmental hazard system

Design 1: Create a Fire class that extends Ground and takes the particular Ground reference as a parameter to keep track of the current tile that is on fire. The actual Ground reference is passed to Fire by getting the current location of the actor that is holding the lantern. A GroundAbility enum would be used to specify which grounds are flammable.

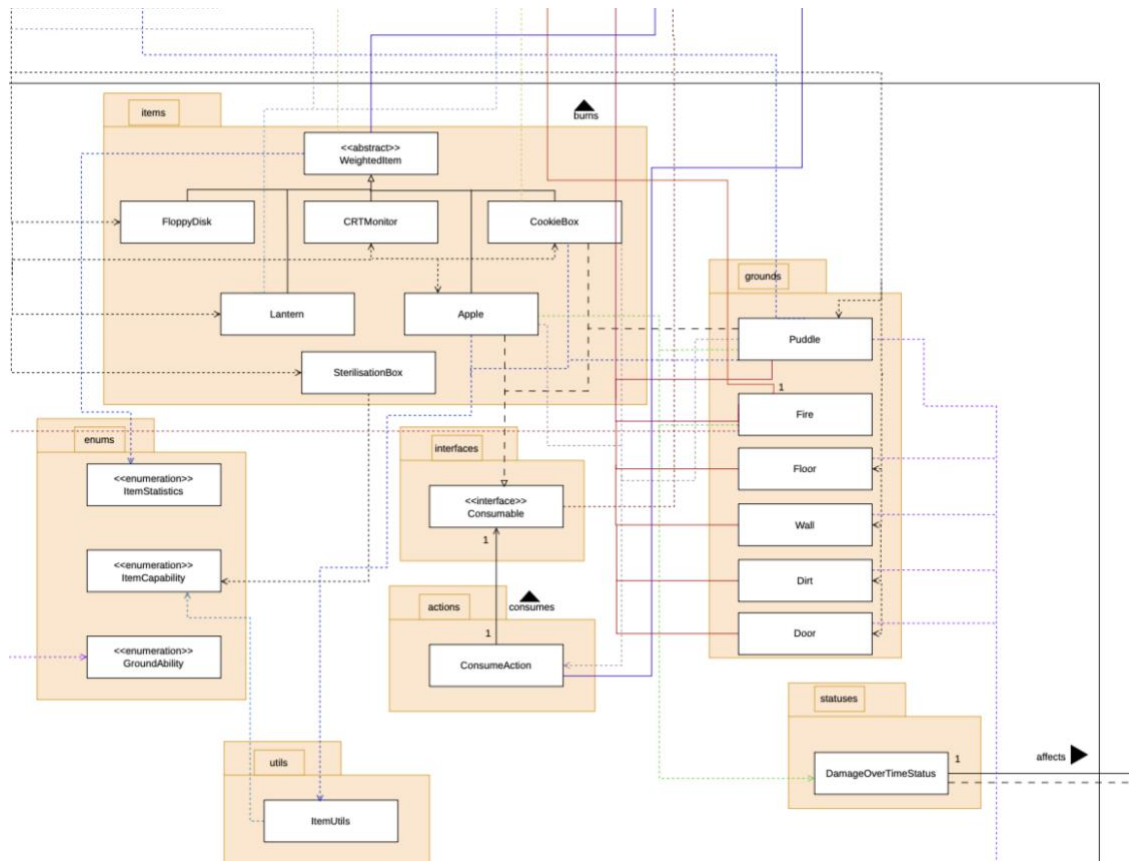
Pros	Cons
Makes it easy to apply effects step on to a fire tile and creates clear separation between fire and normal ground tile.	Requires keeping track of each ground entity, which involves replacing and restoring the specific ground tile.
Ensures the actual ground entity is being passed to Fire instead of manually checking whether each ground is on fire in each of the ground classes.	

Design 2: Create a Flammable interface and add a burning state to each Ground class. Each Ground class implements the Flammable interface and maintains its own burning state (eg. isBurning).

Pros	Cons
No need to replace or modify map tiles dynamically.	Requires modifying every ground class to support burning logic.

Fire state is directly stored in the affected tile.

Duplicates similar logic across multiple Ground implementations.



The diagram above shows the final design of requirement 2, which utilizes each Design 1 of parts A, B and design of part C from the design above to implement safe consumption, have a unified status effect system and have a modular system to deal with the oil leakage on the grounds.

Starting off, having a utility method for the Boolean method hasSterilisationBox() ensures Interface Segregation Principle is not violated, as adding this method to Consumable would mean all items implementing this interface would have to force its way to make use of the method (eg. FirstAidKit). Besides that, new safety checks can be added without modifying existing consumable items (Open-Closed Principle).

Furthermore, a single DamageOverTimeStatus class is created to handle all damage over time effects, including poison from Apple and Puddle and burning effect from the Fire.

Rather than creating separate PoisonStatus and BurningStatus classes, DamageOverTimeStatus accepts variable values for the damage amount and number of turns, allowing it to be reused across all the effects. This eliminates duplicated status logic across multiple classes, adhering to the DRY Principle. Each instance of DamageOverTimeStatus is solely responsible for tracking and applying its own damage effect, making it adhere to the Single Responsibility Principle. Besides that, new damage per turns effect can be introduced simply by creating a new instance with different parameters, without modifying the existing DamageOverTimeStatus, satisfying the Open-Closed Principle.

Lastly, the Fire class extends Ground and takes the original Ground reference as a parameter, storing it so that the actual ground can be restored once the fire has expired. A GroundAbility enum is used to mark which ground tiles are flammable, avoiding any instance of checks or direct coupling to specific ground classes. This results in low coupling between Lantern, Fire and individual ground classes as Lantern only needs to check if the current ground has the FLAMMABLE ability before spawning a Fire. Since Fire extends Ground and can be placed on any map tile in place of the original ground, it behaves correctly as a ground tile whilst also applying its burning effect (Liskov Substitution Principle).

The final design of Requirement 2 extends the item and environment system by introducing safe consumption handling, a unified damage over time status system and a modular environmental hazard model. These components work together to ensure consistent behaviour across consumables, status effects, and ground interactions while avoiding duplicated logic. The design emphasises reuse and maintainability by centralising shared behaviour and keeping responsibilities clearly separated.

REQ 3

A. Implementation of Creature Spawning System

Design 1: Add logic to a Hole class that extends Ground and checks every turns if there isn't an actor on the hole, in which case it has a 50% chance of spawning 2 different creatures.

Pros	Cons
Each hole on the map handles its own spawning logic and ensures a fair chance of spawning a creature at that hole.	Each spawning tile repeats similar logic.
Spawn location is tied to the map location.	

Design 2: Modify EclipseNebula to have a ticking system, where it scans all the holes on the map and decides what to scan and where.

Pros	Cons
Easier to manage ticks across all holes in one go.	EclipseNebula performs too much functionality.
	Since the logic is across every hole on the map and not per hole, based on the percentage chance, only one creature is spawned in all holes.

B. Implementation of Creature class design

Design 1: Create 2 separate creature classes Undead and Slime, which extend Actor. Both classes implement their own set of behaviours.

Pros	Cons
Each creature behave differently and are self-contained. There is a clear separation of each of their behaviours.	Shared logic (such as WanderBehaviour) are duplicated.
Don't need shared abstraction.	More creatures would result in more overlaps in setup.

Design 2: Create a shared base class Creature which would need to be extended by Undead and Slime.

Pros	Cons
New creatures created can just extend this base class.	Introduces additional layer of abstraction, with this base class becoming too general.
Reduces the need for multiple setup codes.	May force unnecessary shared structure as Undead and Slime behave very differently.

C. Determining valid attack targets

Design 1: Create ActorType enum and give the ContractedWorker the ability WORKER. Undead will then check if the actor has this ability and perform AttackAction accordingly.

Pros	Cons
Easy to extend and maintain by adding new abilities for actors.	Need to ensure correct ability is given to each actor.
Avoids direct type checking using instanceof.	Conditional checks are needed to verify the abilities.
Reusable across multiple actor behaviours.	

Design 2: Use instanceof to determine whether a particular actor is a worker.

Pros	Cons
Simple and direct to implement.	Requires repeated type-checking logic.
No need an additional enum.	Strong coupling to specific classes.
	Overuse of instanceof is considered bad code smell.

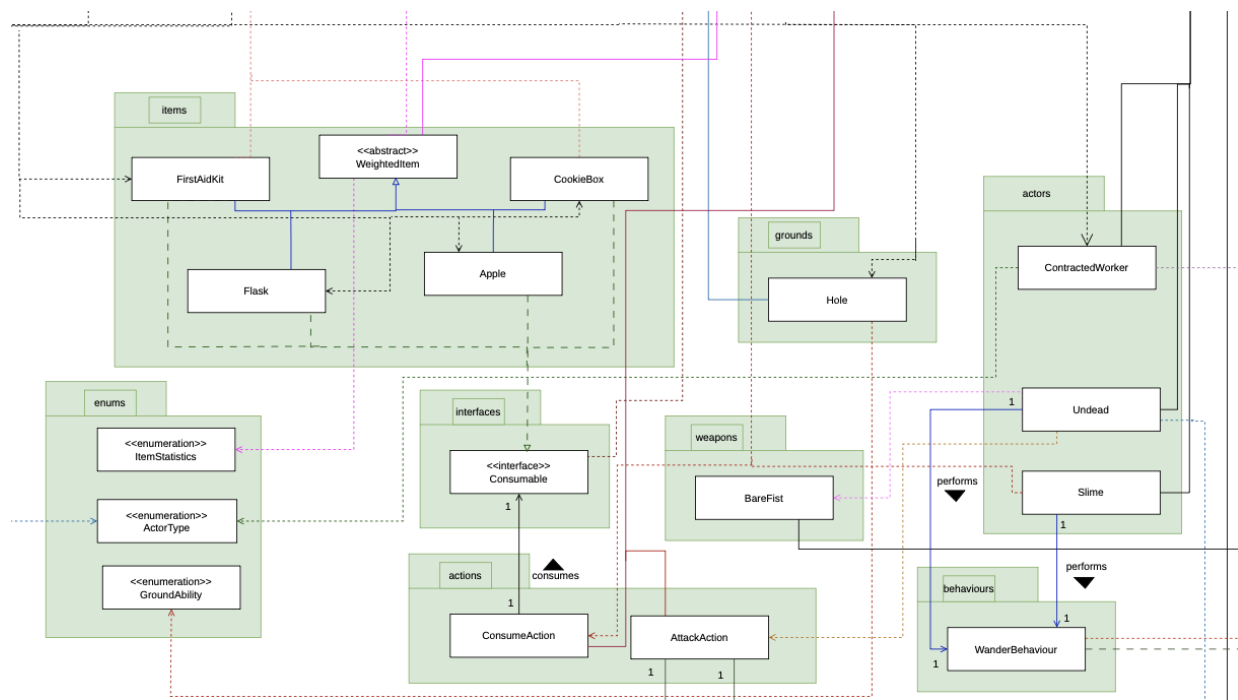
D. Implementation of slime item consumption

Design 1: Reuse existing Consumable and ConsumeAction where slime interacts with these 2 interface and class. Any items consumable by worker can also be consumed by the slime with the same effects applied. The only thing would be slime consumes directly from the ground and doesn't have a sterilization box.

Pros	Cons
Reuses existing logic without duplication and easy to extend as new consumable items are added.	Less flexibility if slime specific behaviour is required.
Ensures consistency between the behaviour of the slime and worker.	The slime behaviour would depend on how Consumable is implemented.

Design 2: Create a separate consumption logic for slime. A different SlimeConsumable interface and SlimeConsumeAction class could be created.

Pros	Cons
Allows custom behaviour specific for slime.	Duplicates existing consumption logic as the items consumed by the slime are exactly those of the workers.
	Harder to maintain consistency across all actors.



The final design of REQ 3 utilises Design 1 of parts A, B, C and D to implement the creature spawning system, valid attack target determination, slime item consumption, and creature class design respectively.

Firstly, the Hole class extends Ground and handles its own spawning logic internally. Each Hole tile on the map independently checks each turn whether it contains an actor and if not, has a 50% chance of spawning one of two different creatures. By keeping the spawning logic self-contained within Hole, each class is responsible for its own behaviour (Single Responsibility Principle). As compared to Design 2 where EclipseNebula would scan all holes and decide what to spawn, keeping the logic in Hole itself ensures EclipseNebula is

not overloaded with responsibilities, which helps ensure low coupling between the world initialization class and the creature spawning system.

In addition, Undead and Slime are implemented as separate concrete classes that both extend the abstract Actor class provided in the engine. Rather than introducing a shared Creature class, each creature class defines its own behaviours independently, keeping the design simple and avoids forcing unnecessary shared structure between two creatures that behave very differently (KISS). Both Undead and Slime correctly extend Actor and can be used anywhere an actor is expected without breaking the existing behaviour (Liskov Substitution Principle).

Moreover, ActorType enum is introduced to determine valid attack targets for the hostile creature rather than using instanceof. ContractedWorker is assigned the ability WORKER and Undead checks for this ability to determine whether it performs AttackAction on the Actor or not. This can be seen on the UML by the dependency. This removes directly coupling between the Undead and ContractedWorker class, resulting in low coupling. This allows for new actor types to be introduced and marked as valid targets simply by assigning the appropriate ActorType ability without modifying Undead or any other existing classes, which follows the Open-Closed Principle.

Lastly, the existing Consumable interface and ConsumeAction class are reused for Slime item consumption. This eliminates the need for separate versions of these files specifically for Slime. Since the items consumed by slime are exactly those consumed by workers, with the same effects applied, duplicating the consumption logic would violate the DRY Principle. By depending on the Consumable abstraction, ConsumeAction works seamlessly for both worker and slime consumption, which demonstrates the Dependency Inversion Principle.

The final design of Requirement 3 introduces a scalable creature system through localised spawning, independent creature classes, modular behaviour composition, and reuse of existing consumable interactions. This approach allows different creatures to exhibit distinct behaviours while sharing common systems such as actions and effects. Overall, this design supports extensibility and behavioural diversity without introducing unnecessary abstraction or duplication.

REQ 4

A. Alarm system state management

Design 1: Create a dedicated AlarmStatus class that implements Status. This class is meant to tell when the alarm is triggered and is active, and the state of the door (whether it is unlocked or not).

Pros	Cons
Encapsulates all alarm related logic in one place and keeps responsibilities separated from other classes.	Introduces global shared state, which increases coupling between classes
Easy to extend if new trigger effects are added.	Classes depend directly on EclipseNebula to access alarm data

Design 2: Use enum AlarmActivity to represent alarm state.

Pros	Cons
Simple and straightforward implementation.	Harder to manage complex logic such as timers and multiple effects.
Can easily add more abilities related to the state of the alarm.	Requires additional structure to store and update the enum state.

B. Alarm triggering mechanism

Design 1: Ground based trigger with a MouseTrap class that extends Ground. Alarm gets triggered when a worker is standing directly above the tile.

Pros	Cons
Easy to trigger based on worker movement and location.	Less flexible for other types of triggers.
Keeps trigger logic localized to the ground.	Requires modification or creation of specialized ground types.
Simple implementation.	

Design 2: Item based trigger where worker picks up the item and adds the item to the inventory.

Pros	Cons
Reuses existing item interaction system.	Trigger depends on specific item interaction.

Easy to integrate with inventory logic.	Requires additional checks during item pickup.
---	--

C. Door locking as a result of alarm being triggered

Design 1: Door checks AlarmStatus.

Pros	Cons
Only involves overriding the canActorEnter() and allowableActions() method in the Door class with checks to tell if alarm is active.	Door requires external alarm system AlarmStatus by directly accessing EclipseNebula.
Easy to apply locking action to all doors without modifying actors.	
Makes use of Boolean methods from AlarmStatus.	

Design 2: ContractedWorker handles door restrictions where the worker decides whether it can move into a door location.

Pros	Cons
Keeps door class simple and unchanged.	Movement rules are duplicated across different actors.
Easy to customize behaviour per actor type.	Breaks abstraction of environment controlling access rules.

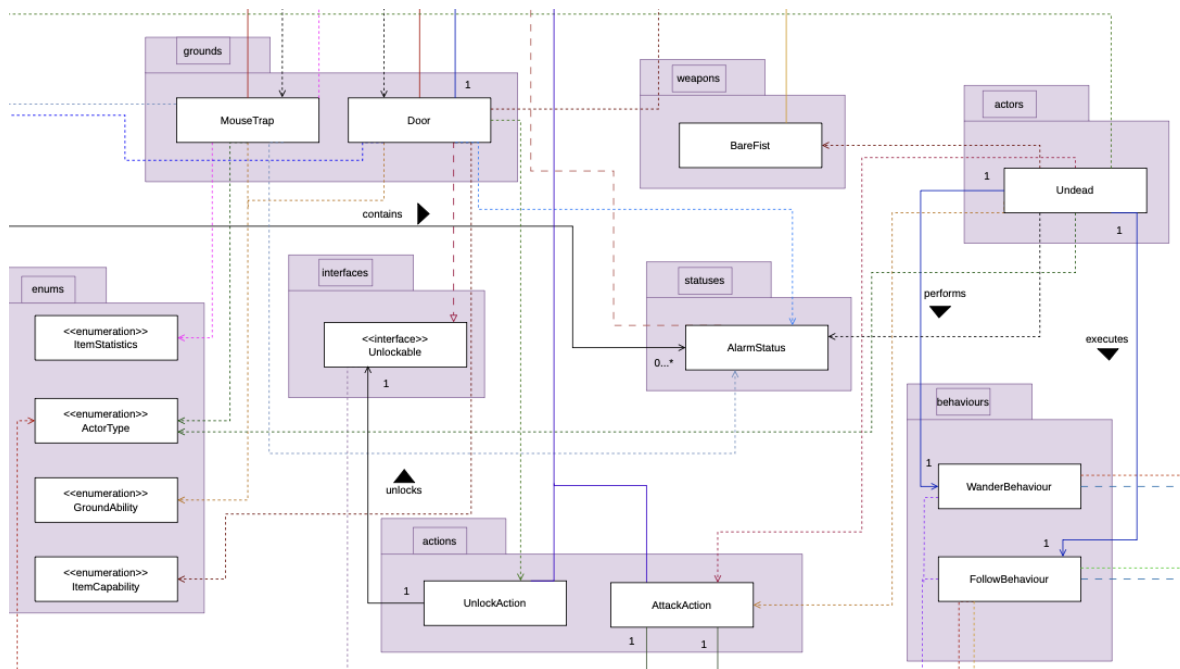
D. Undead movement behaviour as a result of triggering the alarm

Design 1: Create a separate FollowBehaviour class that extends Behaviour.

Pros	Cons
Ensures Undead is only in charge of when to use which behaviour.	Requires additional behaviour related class to be created.
Can be used with other actor/creature entities in the future (eg. Drone).	
Easy to extend with new behaviours.	

Design 2: Follow logic is implemented in the playTurn() of Undead itself.

Pros	Cons
No need for the introduction of new classes.	Movement becomes tightly coupled to the Undead class.
Easier to understand movement logic in one place.	Harder to reuse logic for other entities.
	playTurn() becomes large and harder to maintain.



The final design of REQ 4 utilises Design 1 of parts A, B, C and D to implement the alarm state management, alarm triggering mechanism, door locking consequence and undead chasing consequence.

The AlarmStatus class is introduced to manage all alarm state related logic, including whether the alarm is active or not, and if the door is locked. Rather than scattering alarm state across multiple classes or using a simple enum, encapsulating all alarm logic within

a single `AlarmStatus` class ensures this class has a single purpose (Single Responsibility Principle). The `AlarmStatus` instance is stored per map in `EclipseNebula` using a `HashMap<GameMap, AlarmStatus>`, accessed through a static `getAlarmStatus(GameMap map)` method. This design ensures that each moon has its own independent alarm state rather than sharing a single global alarm, correctly scoping the alarm to its respective map. New alarm related state or behaviour can be added to `AlarmStatus` without modifying any other classes that depend on it (Open-Closed Principle).

Other than that, the `MouseTrap` class extends `Ground` and functions as the trigger for the alarm. When the worker steps directly on the `MouseTrap` tile, the alarm is activated by calling `trigger.active()`. By keeping the trigger logic self contained within `MouseTrap`, the triggering mechanism is decoupled from the alarm state management in `AlarmStatus` and the consequences are handled in `Door` and `Undead`, which results in low coupling between these classes. Additionally, `MouseTrap.tick()` is responsible for ticking the `AlarmStatus` countdown each turn when the alarm is active, keeping the ticking logic localised without requiring modifications to any existing engine classes (KISS).

Furthermore, door locking logic is implemented by overriding the `canActorEnter()` and `allowableActions()` method in the `Door` class to check the `AlarmStatus` directly. Compared to design 2 where the `ContractedWorker` handles the door locking on its own, keeping the logic within `Door` ensures that the environment controls its own access rules rather than delegating that responsibility to actors, maintaining Single Responsibility Principle and high cohesion within `Door`. The `currentLocation` is stored during `tick(Location location)` and reused in `canActorEnter()` to access the map and retrieve `AlarmStatus`, keeping the implementation simple without introducing additional parameters or modifying the engine, which is in adherence to KISS.

Lastly, the `Undead` chasing the worker consequence uses a `FollowBehaviour` class which performs the main logic of moving around the creature one step at a time towards the closest worker, for as long as the alarm is active. This is performed in the `playTurn()` of `Undead`, which uses a priority based approach to ensure it only checks the alarm status and then focuses on the `WanderBehaviour`. `Undead` is essentially only responsible for deciding which behaviour to use rather than implementing movement logic itself, thus complying with the Single Responsibility Principle. In addition, `FollowBehaviour` encapsulates the logic for moving one step closer to a target actor each turn using Manhattan distance calculation, and can be reused by any future hostile entity such as drones without modification, conforming to Open-Closed and DRY Principles. Since `FollowBehaviour` and `WanderBehaviour` both implement the `Behaviour` interface, they can

be swapped at runtime without Undead needing to know the details of either implementation (Dependency Inversion Principle).

In conclusion, the final design of Requirement 4 implements a map based alarm system using a centralized AlarmStatus, alarm triggering using MouseTrap and distributed consequences across Door and Undead behaviour. This ensures that alarm state is managed consistently while allowing different parts of the system to respond independently. The design achieves a balance between centralised state management and modular behaviour, enabling scalable extension of alarm effects.